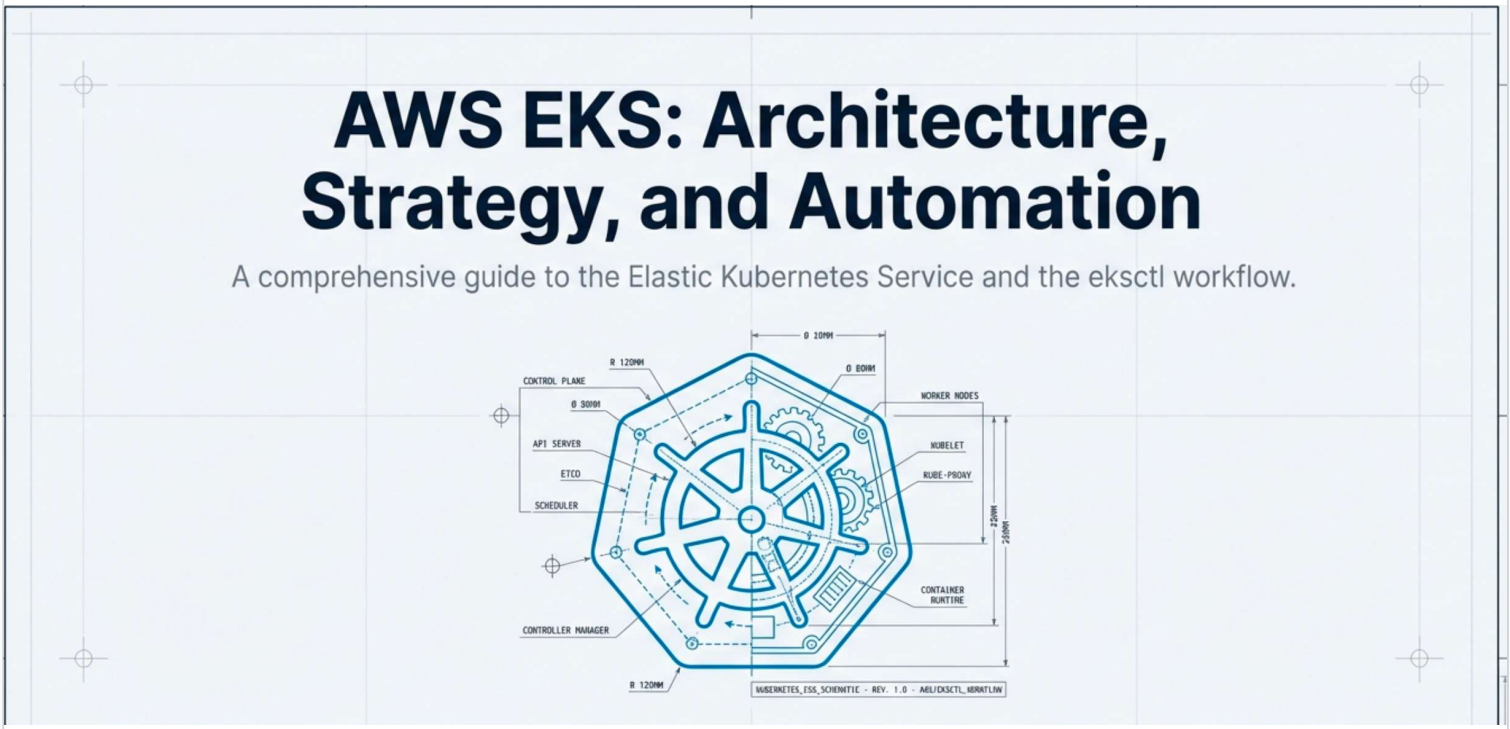
AWS Elastic Kubernetes Service (EKS)

Updated January 19, 2026 • 6 min read • View as Markdown





Narottam Singh Bisht
Cloud & DevOps learner documenting hands-on learning with AWS, Kubernetes, and cloud-native tools....

✅ What Will You Learn?

By reading this article, you will learn:

- What **Amazon EKS (Elastic Kubernetes Service)** is and why it is used

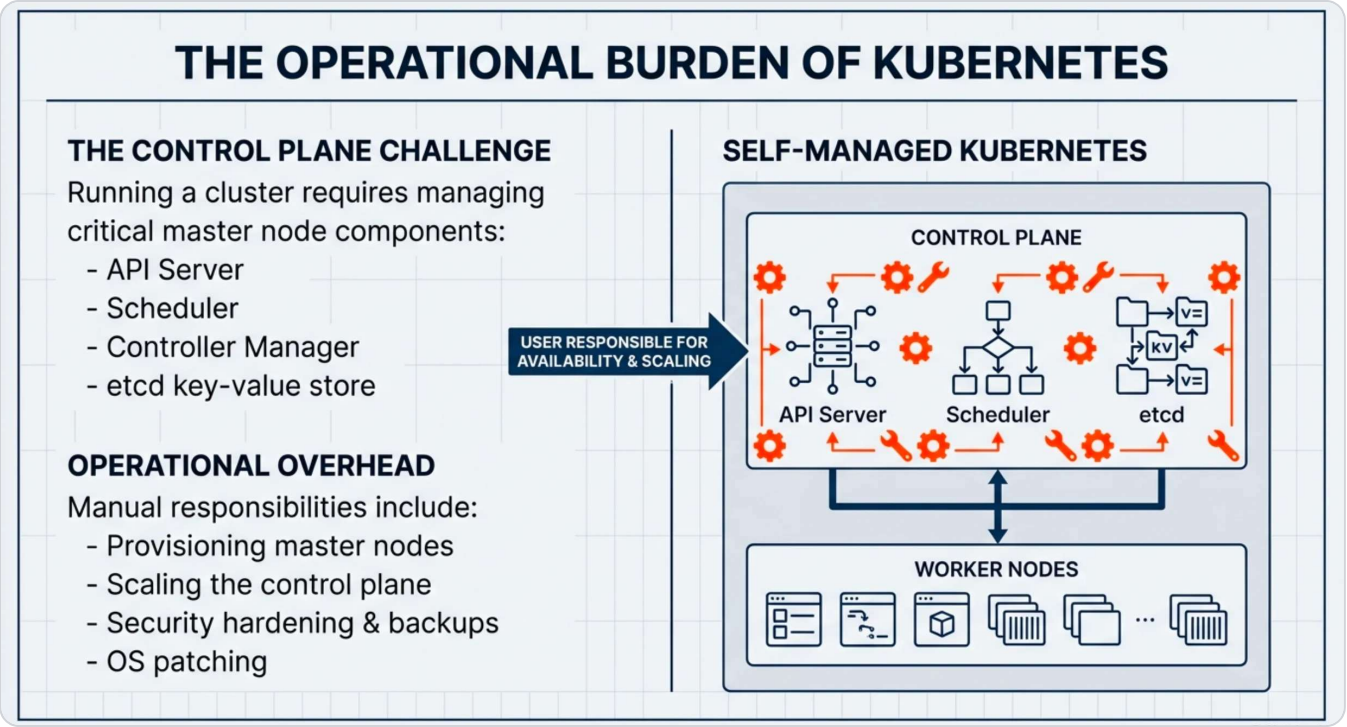
- The core components involved in an **EKS architecture** (cluster, nodes, control plane)
- How **EKS simplifies Kubernetes management** on AWS
- The difference between **managed Kubernetes (EKS)** and self-managed Kubernetes

What is AWS EKS?

AWS EKS is a managed Kubernetes service, where AWS takes responsibility for the control plane (master nodes). This includes provisioning and maintaining the API server, scheduler, controller manager, and `etcd`. As a user, you only need to manage worker nodes.

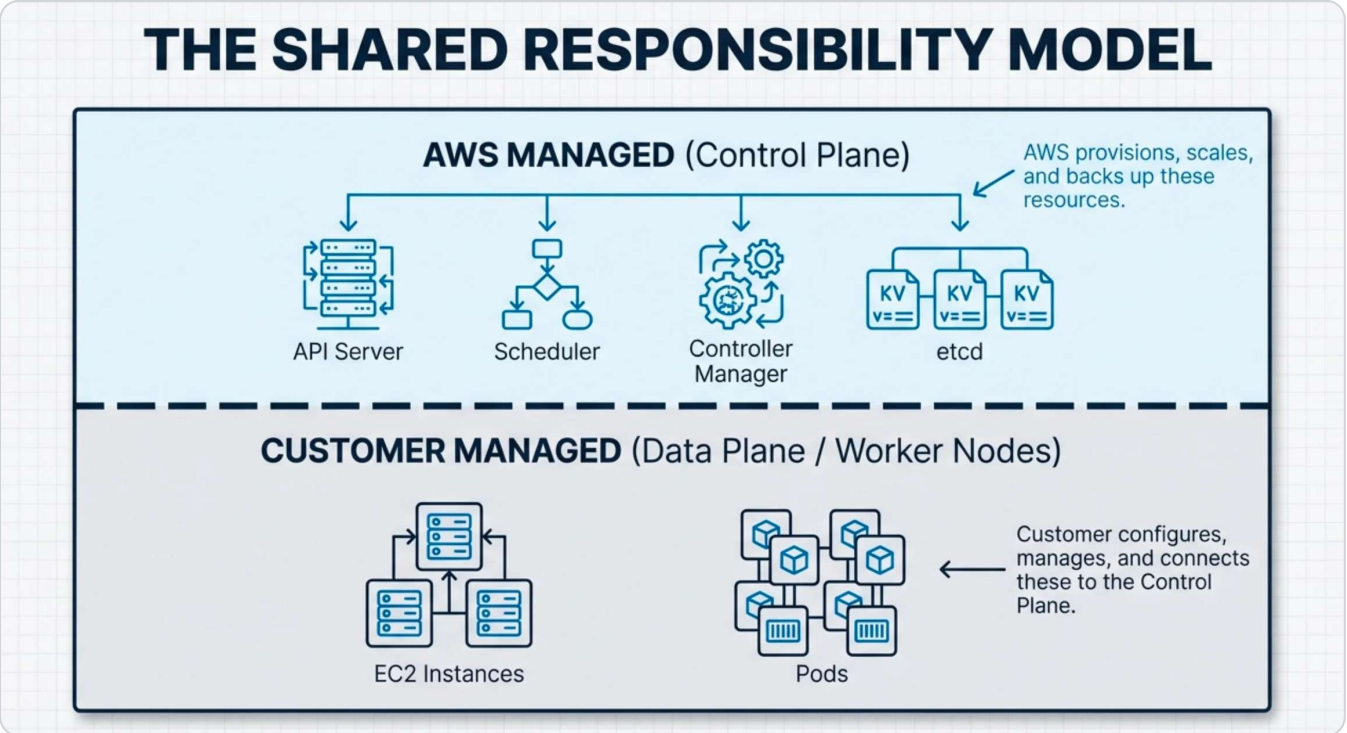
Why not self-managed Kubernetes cluster?

There is a lot of overhead to creating, managing, scaling, and maintaining a self-hosted Kubernetes cluster. We can offload all that work to a managed AWS EKS cluster. You are bound to have issues if you are managing and setting up the cluster by yourself. You have to take care of the securing your self-managed Kubernetes cluster.

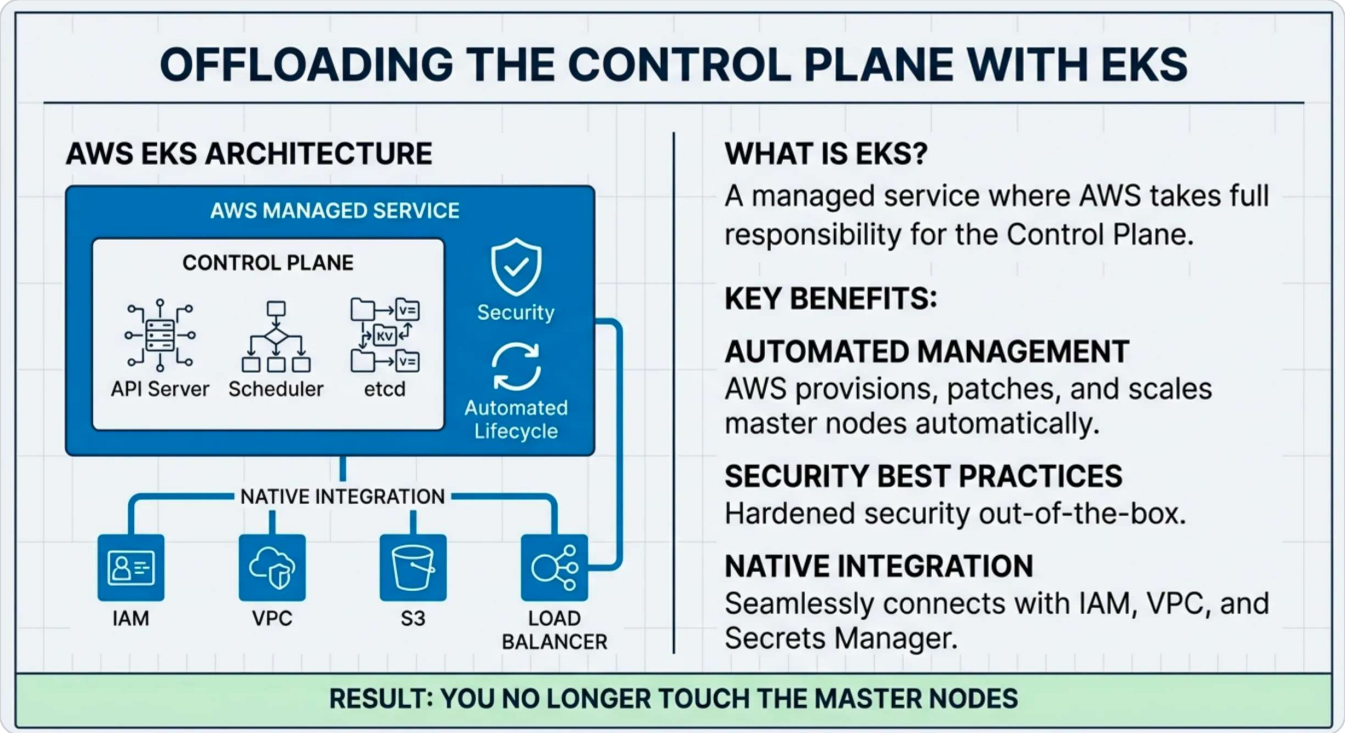


Why EKS?

AWS EKS works on a shared responsibility model, where EKS manages the control plane and the customer manages worker nodes. There are different services that manage the worker nodes for you in AWS, which is discussed below.



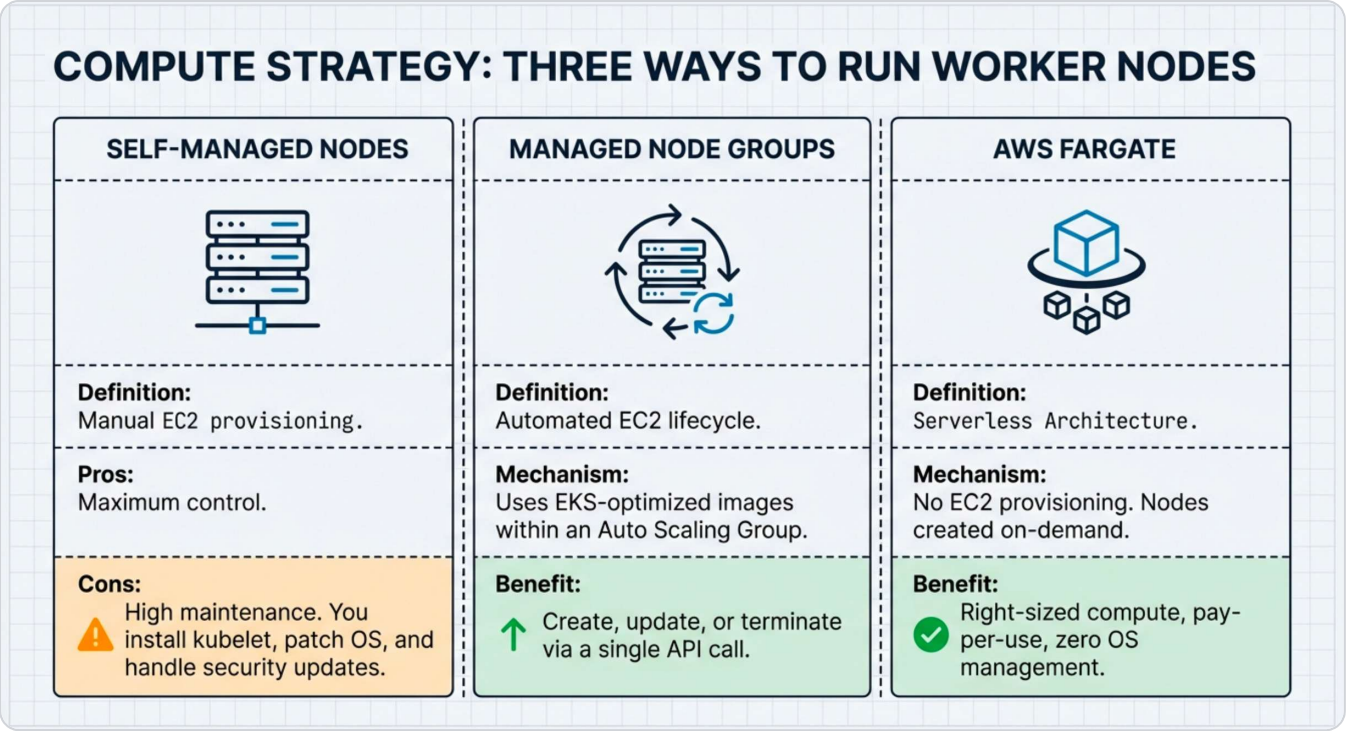
The benefits of using AWS EKS are many because of its widely accepted way of running and managing clusters. Customer only has to be concerned about the usage of cluster not the management part, which is mostly taken care of by AWS. And using AWS EKS cluster gives you the benefit of easy coupling with other AWS services, which will be a hassle otherwise.



1. Reduced Complexity: Managing Kubernetes yourself is difficult and time-consuming.
2. Security: AWS follows best practices for security and cluster hardening.
3. Integration: Seamlessly integrates with other AWS services like S3, IAM, and Secrets Manager.

Worker Node Management Options

Since EKS only manages the control plane, it’s up to you to setup worker nodes. You must choose a method for these worker nodes:



1. **Self-managed Nodes:** You manually provision, configure (install kubelet, container runtime), and patch EC2 instances and register them with your cluster's control plane yourself.

2. **Managed Node Groups:** AWS automates the provisioning and lifecycle management of EC2 instances using EKS-optimised images. Every node is part of an Auto Scaling Group (ASG) that's managed for you by EKS.

3. **AWS Fargate:** A serverless option where you don't provision EC2 instances at all. Fargate creates worker nodes on demand based on your container's resource requirements. That's the most pocket friendly and easiest option in my opinion. It automatically creates the right EC2 instance based on your requirement and is pay-per-use. It automatically scales up and scales down based on the demand.

Why to move from Console to Code?

In order to make EKS work from the AWS console, you have to give:

1. Container name for the cluster and k8s version that you want to run.

2. Setup IAM role for the cluster, so it has all the necessary privileges to perform the various operations it needs like provisioning nodes, accessing secrets and storage.

3. Select VPC & subnets where you want your EKS cluster running.

4. Define security group for cluster: to lock down what traffic goes to and from your cluster.

5. Then, you have to create worker nodes for which you have to:

    1. Create Node group: It's nothing but a group of nodes that you'll register in your control plane.

    2. Select instance type that you want to use for your EC2 instances.

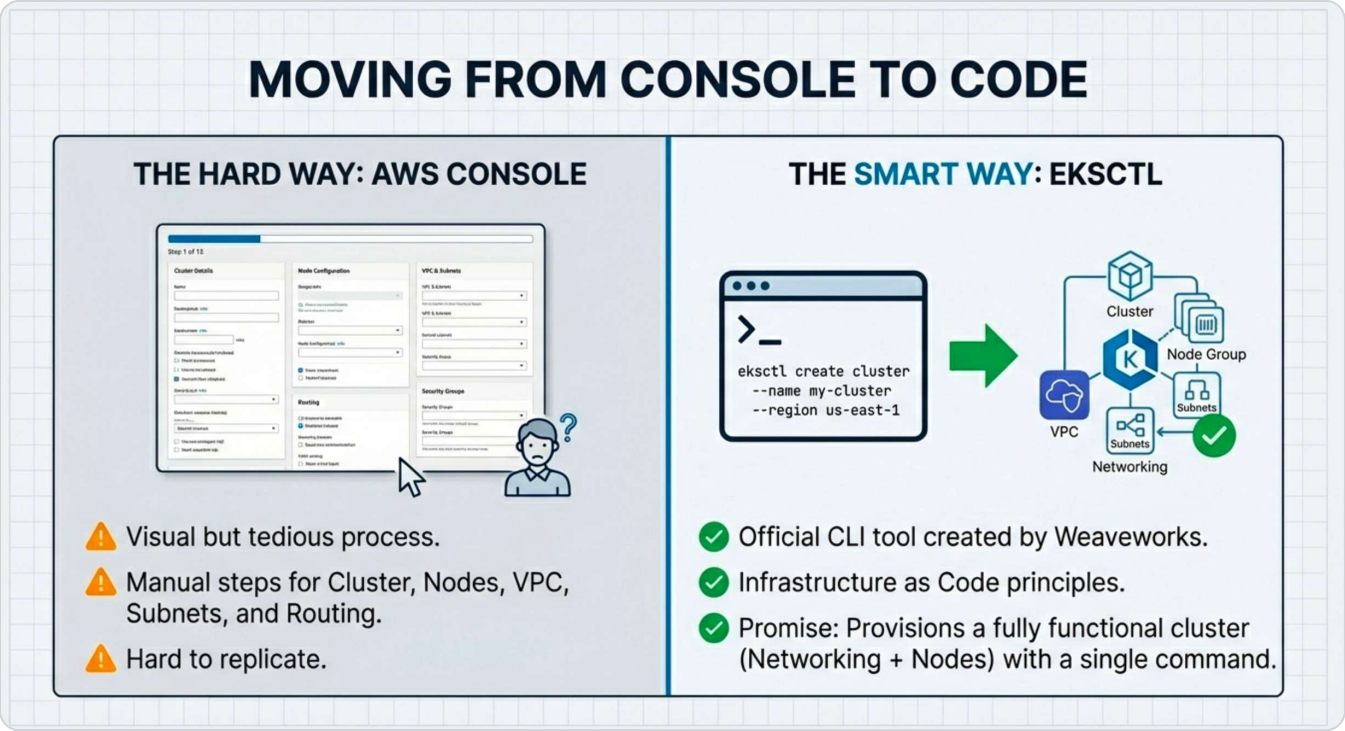
    3. Define the min/max number of nodes that you want in your node group.

    4. Specify EKS cluster to connect to, so here, we'll specify the EKS cluster that we setup in the previous step.

6. Then, connect to your EKS cluster from your local machine. For that, go to AWS Console, get the connection information for the EKS cluster, and run set-cluster command, then you'll be able to deploy your application.

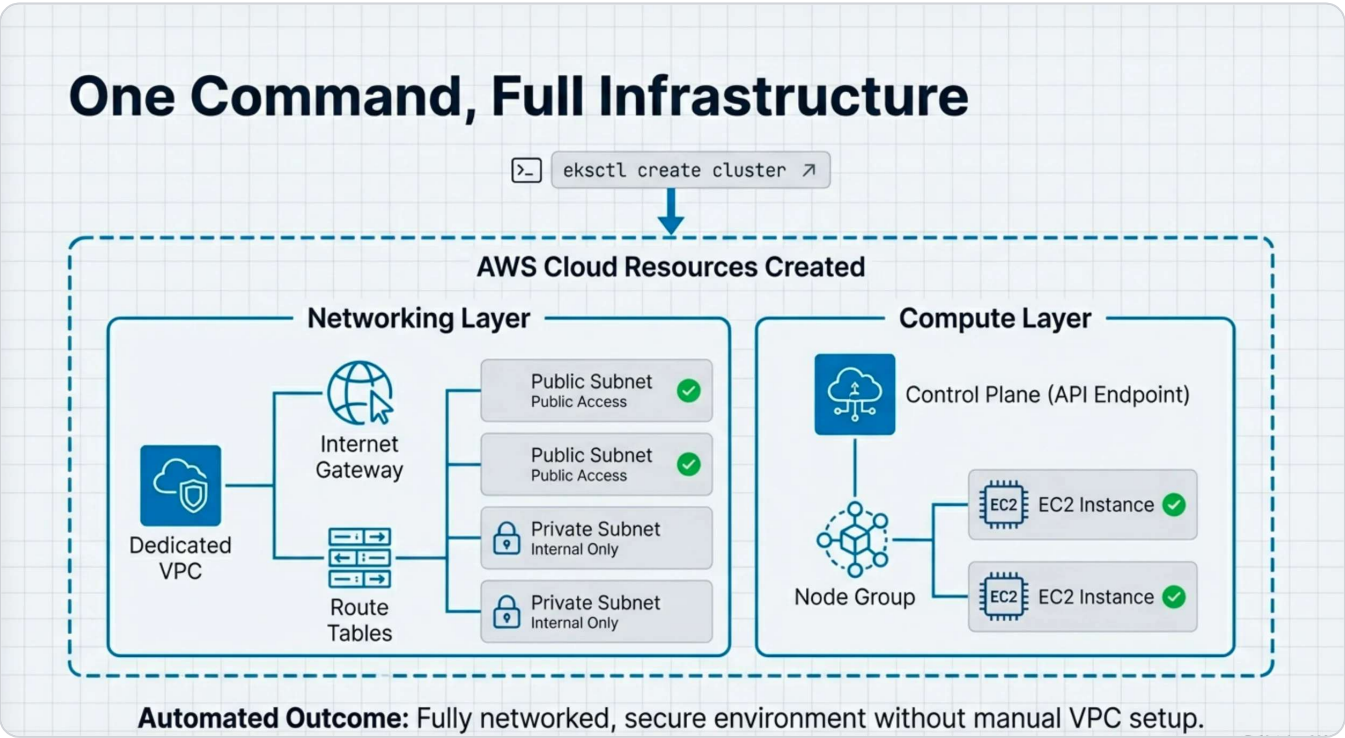
    ```
    kubectl config set-cluster <cluster_id>
    ```

Whereas if you don't want to follow this long and tedious process of creating this cluster yourself and if you are comfortable with the code, you can provision your entire Kubernetes cluster with a single command, and it will automatically create security groups, VPC, etc.



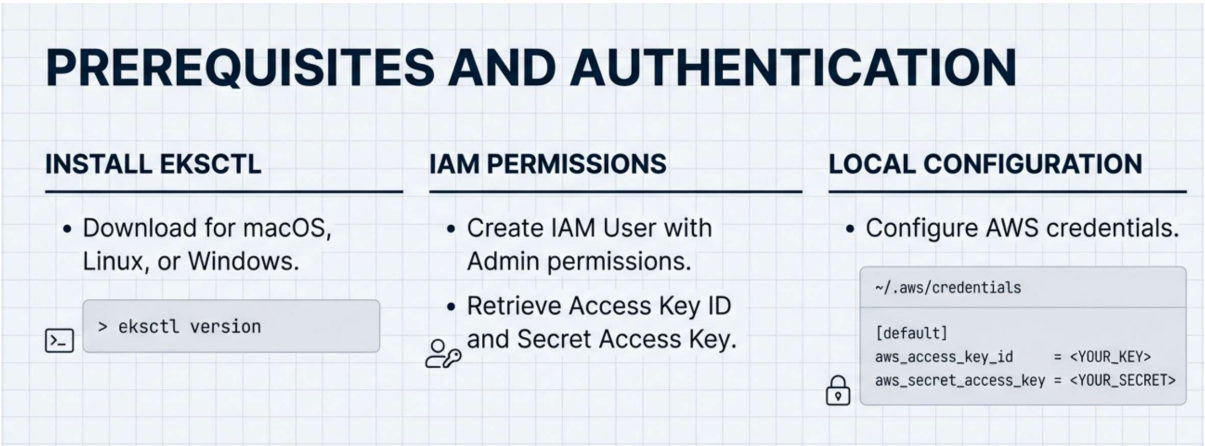
Deploying with `eksctl`

While you can use the AWS Console, the process is tedious. The command-line utility eksctl (created by Weaveworks) automates the entire setup (VPC, subnets, control plane, node groups) with a single command.



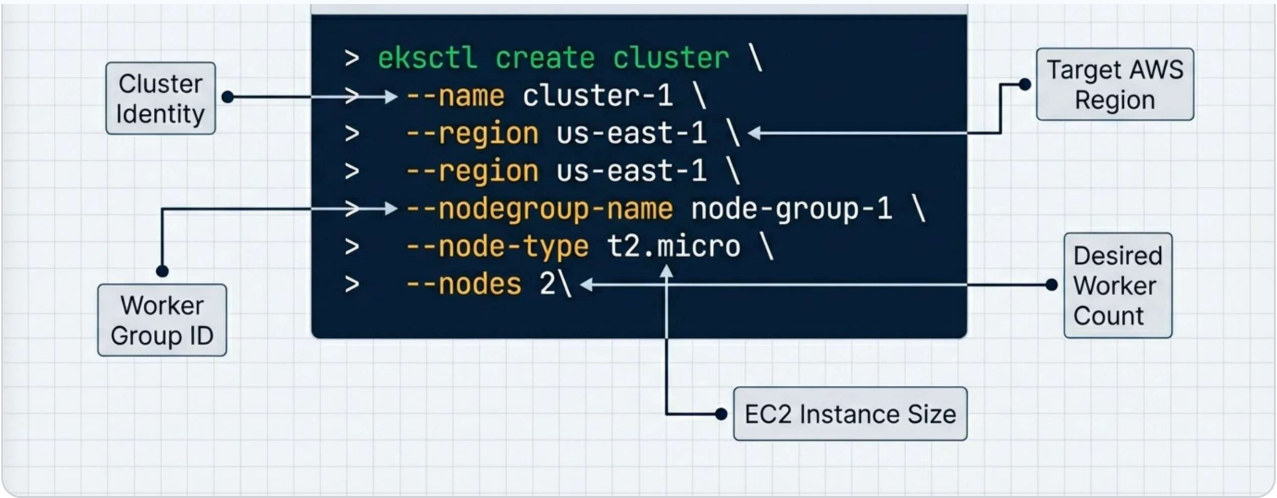
Steps to deploy:

1. Authentication: Set up your AWS credentials (access key and secret key) in the `.aws/credentials` file. To get these credentials and set it up:
    1. Go to AWS IAM → Users → Select a existing user or create new → Security Credentials tab → Under Access Keys section → Create Access key.
    2. If you have the AWS CLI tool, run `aws configure` and it will prompt you to fill the access key and secret.
    3. Follow AWS CLI guide.

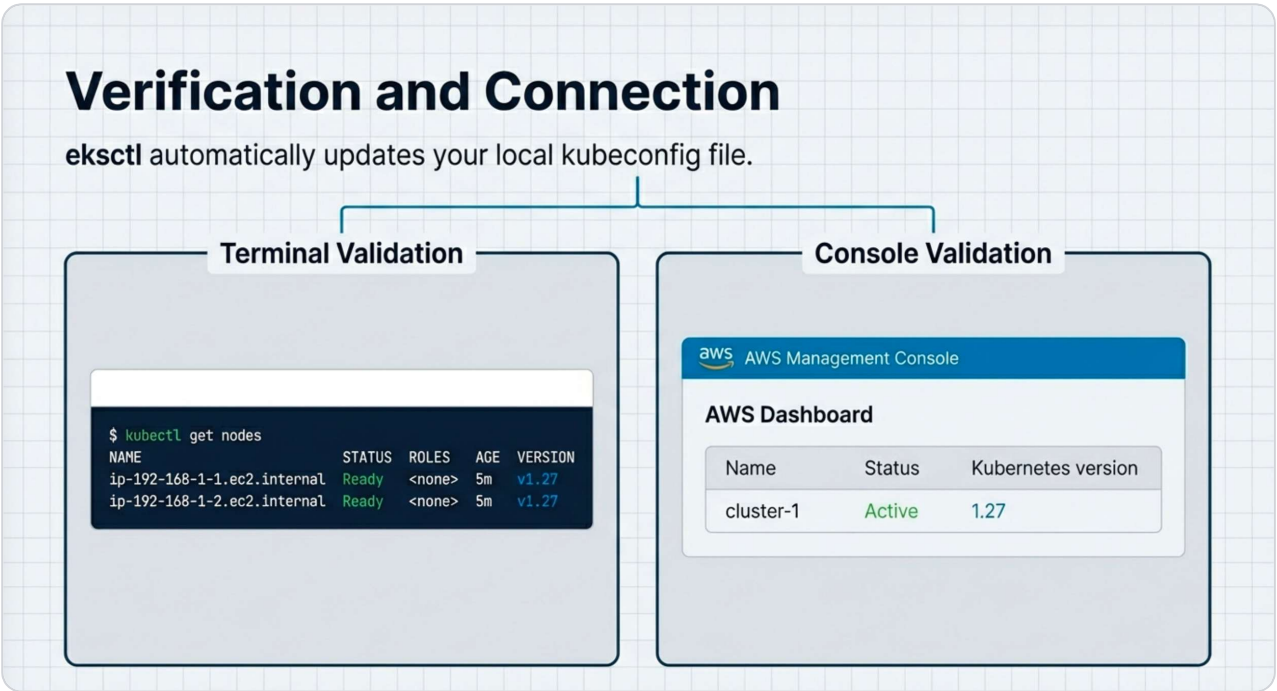


2. Creation Command:

```
eksctl create cluster -n cluster-one --node-group-name node-g
```



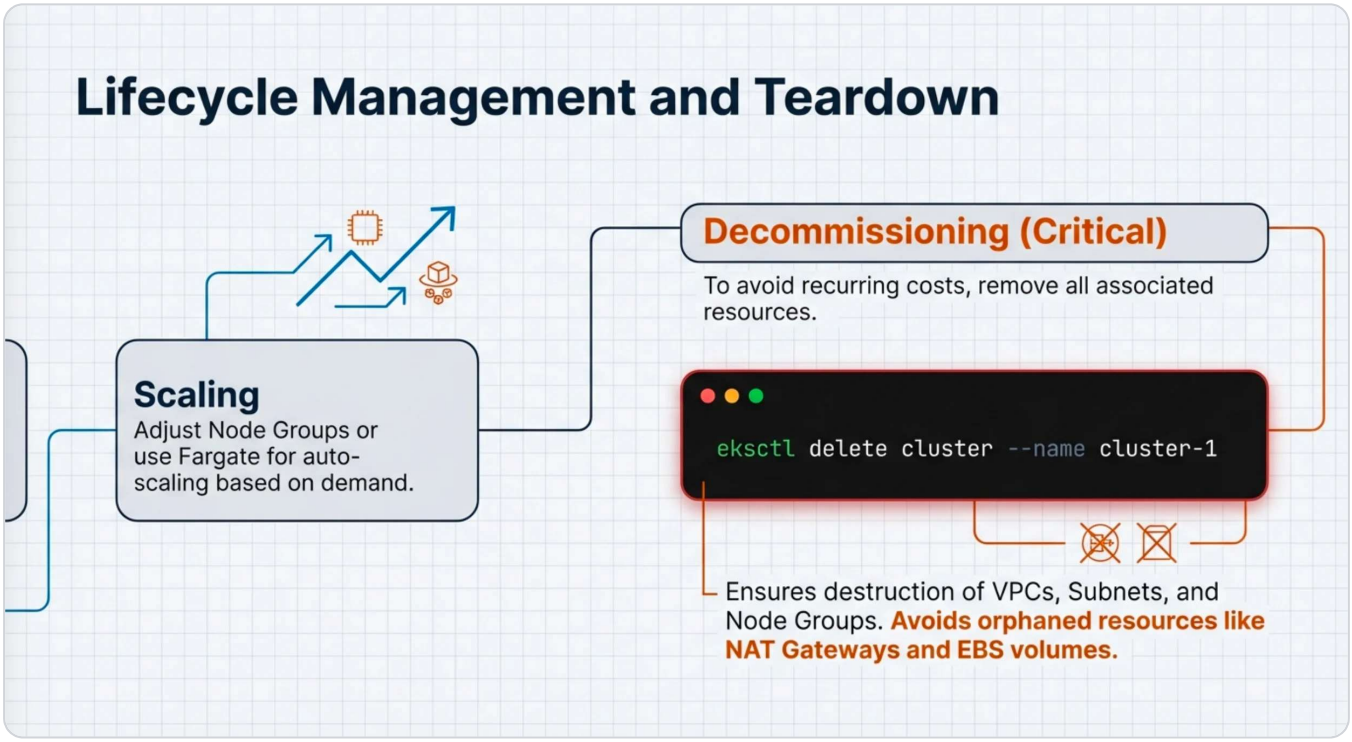
3. **Automatic Config:** `eksctl` automatically updates your local `kubectl` config file, allowing you to run `kubectl get nodes` immediately after the cluster is ready.



Cleaning Up

To avoid ongoing costs, you can delete the cluster and all associated resources (VPC, subnets, etc.) with:

```
eksctl delete cluster -n cluster-one
```



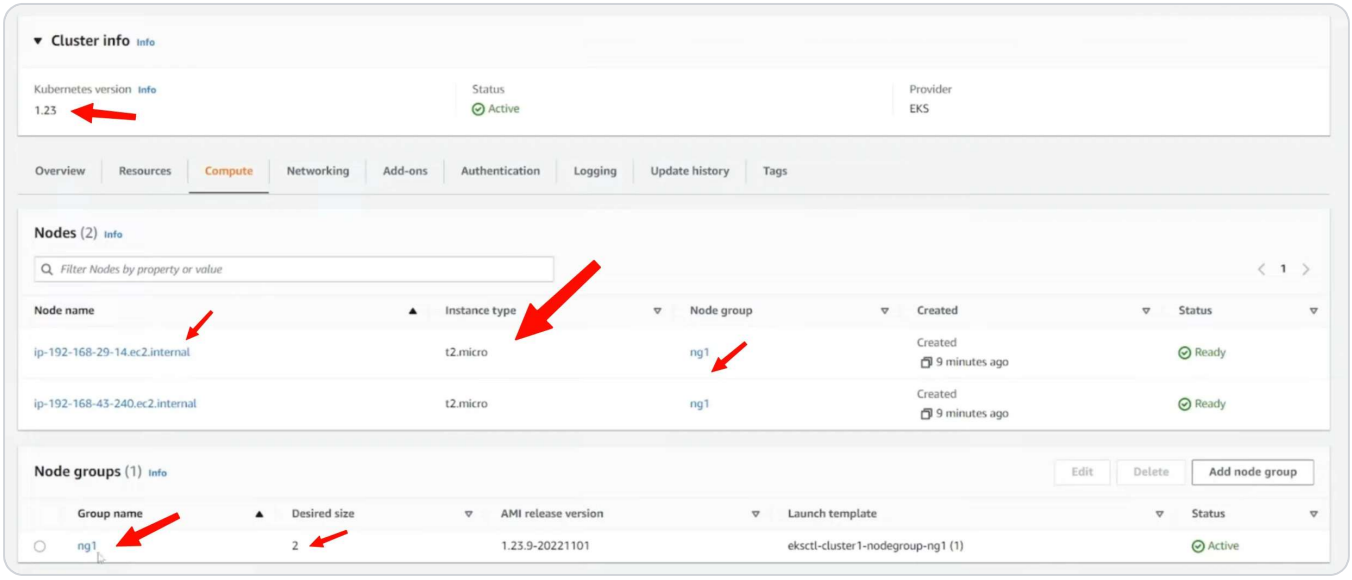
Link to Official EKSCTL documentation

How it looks on AWS Console?

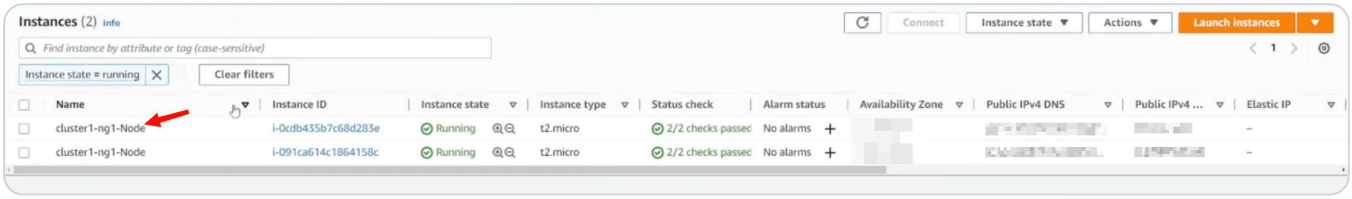
Output of `ekctl create` command:

```
deploying stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
waiting for CloudFormation stack "eksctl-cluster1-cluster"
API server is unreachable
building managed nodegroup stack "eksctl-cluster1-nodegroup-ng1"
deploying stack "eksctl-cluster1-nodegroup-ng1"
waiting for CloudFormation stack "eksctl-cluster1-nodegroup-ng1"
waiting for CloudFormation stack "eksctl-cluster1-nodegroup-ng1"
waiting for CloudFormation stack "eksctl-cluster1-nodegroup-ng1"
waiting for CloudFormation stack "eksctl-cluster1-nodegroup-ng1"
waiting for CloudFormation stack "eksctl-cluster1-nodegroup-ng1"
waiting for the control plane to become ready
saved kubeconfig
no tasks
all EKS cluster resources for "cluster1" have been created
nodegroup "ng1" has 2 node(s)
node                                    is ready
node                                    is ready
waiting for at least 2 node(s) to become ready in "ng1"
nodegroup "ng1" has 2 node(s)
node                                    is ready
node                                    is ready
kubectl command should work with                                    try 'kubectl get nodes'
EKS cluster "cluster1"                          is ready
```

This is how your EKS cluster looks on AWS Console. Nodes (with its instance type, node group and status), Node groups (with its node group name and desired instace size that will scale up and down based on your traffic), Cluster info (with Kubernetes version running on your cluster).



These are the 2 EC2 instances that are created and managed by EKS, which are registered as nodes in EKS cluster.



To get your cluster info, you can run `kubectl config view` and see that your cluster is added to your Kubernetes config, and under context, you'll see the info of your cluster.

```
[redacted] via v1.17.7 via v14.17.3 via v3.9.5 (venv)
❯ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: [redacted]
  name: [redacted]
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: [redacted]
  name: [redacted]
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: [redacted]
  name: docker-desktop
- cluster:
    certificate-authority: ..\Documents\eks1.ca.crt
    server: [redacted]
  name: eks1
- cluster:
    certificate-authority: [redacted]
    extensions:
    - extension:
        [redacted]
        provider: minikube.sigs.k8s.io
        version: v1.25.1
      name: cluster_info
    server: [redacted]
  name: minikube
contexts:
- context:
    cluster: [redacted]
    user [redacted]
  name: [redacted]
- context:
    cluster: [redacted]
```

Output of `kubectl get nodes`

```
[redacted] via v1.17.7 via v14.17.3 via v3.9.5 (venv)
❯ kubectl get nodes
NAME                                        STATUS   ROLES    AGE   VERSION
[redacted]                                  Ready    <none>   12m   v1.23.9-eks-ba74326
[redacted]                                  Ready    <none>   12m   v1.23.9-eks-ba74326

[redacted] via v1.17.7 via v14.17.3 via v3.9.5 (venv)
❯
```

Key Takeaways

- Amazon EKS is a **fully managed Kubernetes service** that reduces the operational overhead of running Kubernetes clusters
- AWS manages the **control plane**, allowing you to focus more on deploying and scaling applications
- EKS integrates seamlessly with AWS services like **IAM, VPC, CloudWatch, and Load Balancers**
- It provides **high availability, security, and scalability** for containerized workloads
- Understanding EKS is an essential skill for **DevOps and Cloud Engineers**
- EKS is widely used in production environments for **microservices and container orchestration**

#devops #aws #kubernetes #docker #containerization

62 views

Comments

Join the discussion

No comments yet. Be the first to comment.

More from this blog

Learn Linux

What Is Linux? Linux is everywhere — from cloud servers to containers.While learning DevOps basics, I realized I didn't truly understand how Linux manages...

Feb 2, 2026 · 24 min read · 37





Journey to the Cloud

2 posts

Journey to the Cloud is my learning-in-public blog where I explore DevOps and Cloud. I share notes, and deep dives into Cloud, If you're moving toward the cloud, you'll find practical takeaways here.

© 2026 Journey to the Cloud